

Wrestling Guide – Python Backend API Development

1. Project Overview

Build the backend/API for a mobile application called **Wrestling Guide**.

The mobile application will be developed separately by another mobile development team. Your responsibility is **ONLY the Python backend and REST APIs**.

Do NOT build:

- Flutter/mobile application
- Mobile UI
- Coach mobile screens
- Student mobile screens
- AI computer-vision model yet
- Video movement analysis yet
- Coach approval workflow yet

Those will be implemented in later phases.

The immediate goal is to establish a secure, scalable backend foundation and provide the mobile team with fully documented APIs.

2. Technology Stack

Use the following technology stack:

Backend

- Python 3.12+
- Django
- Django REST Framework
- PostgreSQL
- JWT authentication using `djanoestframework-simplejwt`

API Documentation

Use OpenAPI/Swagger documentation.

Recommended:

- drf-spectacular

The API documentation should be available through:

```
/api/docs/  
/api/schema/
```

Configuration

Use environment variables for all sensitive configuration.

Use:

```
.env
```

for local development.

Never hard-code:

- Database passwords
- Django secret key
- JWT secrets
- API keys
- Storage credentials
- AI credentials
- Email credentials

3. High-Level Application

There are two types of users:

```
COACH  
STUDENT
```

Coach

Currently there is **only ONE coach**.

The coach is already stored in the database.

Therefore:

- Coach cannot register.
- Coach can only log in.
- Coach can view profile.
- Coach can edit profile.
- Coach can change password.

There must NOT be a public Coach Registration API.

Student

Students can:

- Register
- Login
- View profile
- Edit profile
- Change password

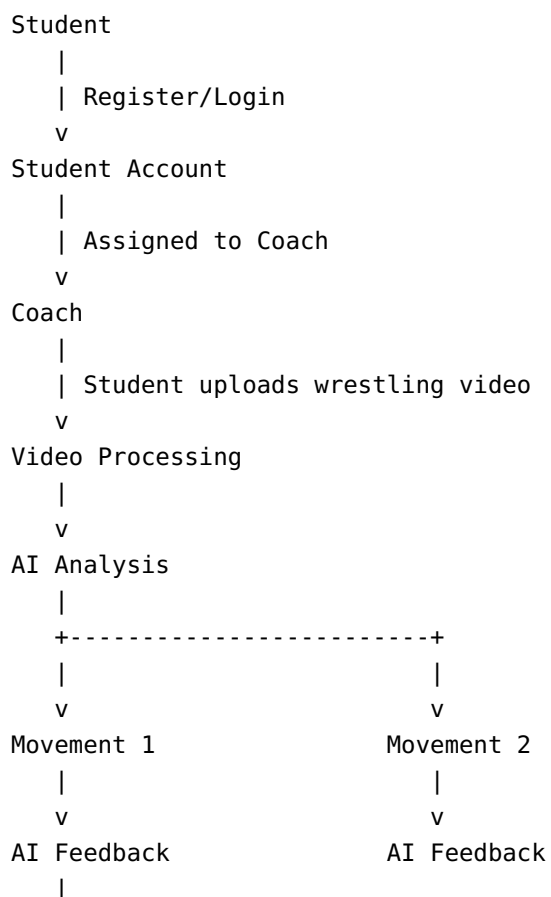
Students will eventually be assigned to a coach.

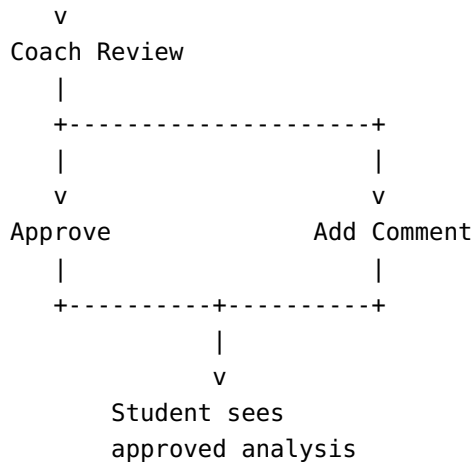
Because there is currently only one coach, new students can automatically be assigned to the existing coach.

However, design the database so that multiple coaches can be supported in the future without major database changes.

4. Future Application Workflow

The backend must be designed with the following future workflow in mind.





The future AI system should eventually identify individual wrestling movements from uploaded videos and provide suggestions.

For example:

Movement:
Single Leg Takedown

Timestamp:
00:12

AI Score:
72

AI Suggestion:
Lower your level before initiating the shot.

Coach Status:
PENDING

The coach can review the AI suggestion.

The coach may:

- Approve it
- Add a comment
- Modify the feedback
- Reject/request improvement

Students should only see analysis after coach approval.

This functionality is FUTURE scope and should not be implemented today.

5. Today's Development Scope

Today's task is ONLY to build the backend foundation and authentication/profile APIs.

Required APIs

Authentication

Coach:

```
POST /api/v1/auth/coach/login/
```

Student:

```
POST /api/v1/auth/student/register/
```

```
POST /api/v1/auth/student/login/
```

JWT:

```
POST /api/v1/auth/token/refresh/
```

Current user:

```
GET /api/v1/auth/me/
```

6. Coach APIs

6.1 Coach Login

Endpoint:

```
POST /api/v1/auth/coach/login/
```

Request:

```
{
  "email": "coach@example.com",
  "password": "*****"
}
```

Successful response:

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "access": "JWT_ACCESS_TOKEN",
    "refresh": "JWT_REFRESH_TOKEN",
    "user": {
      "id": 1,
      "email": "coach@example.com",
      "role": "COACH",
      "first_name": "John",
      "last_name": "Coach"
    }
  }
}
```

Requirements:

- Verify email.
- Verify password.
- Verify user is active.
- Verify user role is `COACH`.
- Do not allow STUDENT users to use the coach login endpoint.
- Do not expose whether the email exists.
- Use generic authentication error messages.

Example:

Invalid email or password.

Do not return:

Email does not exist.

7. Coach Profile

7.1 View Coach Profile

`GET /api/v1/coach/profile/`

Authentication:

```
Authorization: Bearer <access token>
```

Response:

```
{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "id": 1,
    "email": "coach@example.com",
    "first_name": "John",
    "last_name": "Coach",
    "phone": "9876543210",
    "profile_image": null,
    "role": "COACH"
  }
}
```

A coach can only retrieve their own profile.

8. Coach Edit Profile

Endpoint:

```
PATCH /api/v1/coach/profile/
```

Request:

```
{
  "first_name": "John",
  "last_name": "Smith",
  "phone": "9876543210"
}
```

Use PATCH so that users can update individual fields.

Requirements:

- Validate all input.
- Do not allow role changes.
- Do not allow user ID changes.
- Do not allow arbitrary database fields to be updated.
- Email changes should be restricted or handled through a separate secure process.
- Do not allow a coach to create another coach.

9. Coach Change Password

Endpoint:

```
POST /api/v1/coach/change-password/
```

Request:

```
{
  "old_password": "OldPassword123",
  "new_password": "NewPassword123!"
}
```

Requirements:

- Require authentication.
- Verify old password.
- Validate new password.
- Never store plaintext passwords.
- Use Django's password hashing.
- Prevent reuse of the current password.
- Return a generic error when old password is incorrect.

After successful password change, consider invalidating existing refresh tokens/sessions where practical.

10. Student Registration

Endpoint:

```
POST /api/v1/auth/student/register/
```

Request:

```
{
  "first_name": "Rahul",
  "last_name": "Sharma",
  "email": "rahul@example.com",
  "phone": "9876543210",
  "password": "StrongPassword123!"
}
```

Response:

```
{
  "success": true,
  "message": "Student registered successfully",
  "data": {
    "id": 25,
    "email": "rahul@example.com",
    "role": "STUDENT",
    "first_name": "Rahul",
    "last_name": "Sharma"
  }
}
```

Requirements:

- Student role must automatically be `STUDENT`.
- User cannot select their role during registration.
- User cannot register as coach.
- Email must be unique.
- Validate email format.
- Validate password strength.
- Hash password securely.
- Sanitize/validate all submitted data.
- Do not return password.
- Do not return password hash.
- Automatically assign the student to the existing coach if that is the agreed current business rule.

If the coach does not exist, registration should fail safely with an appropriate server-side error and should not create an unassigned account unless explicitly designed to do so.

11. Student Login

Endpoint:

```
POST /api/v1/auth/student/login/
```

Request:

```
{
  "email": "rahul@example.com",
  "password": "StrongPassword123!"
}
```

Response:

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "access": "JWT_ACCESS_TOKEN",
    "refresh": "JWT_REFRESH_TOKEN",
    "user": {
      "id": 25,
      "email": "rahul@example.com",
      "role": "STUDENT",
      "first_name": "Rahul",
      "last_name": "Sharma"
    }
  }
}
```

Requirements:

- Verify credentials.
- Verify role is STUDENT.
- Verify account is active.
- Use generic authentication errors.
- Never expose sensitive authentication details.

12. Student Profile

12.1 View Profile

```
GET /api/v1/student/profile/
```

Authentication required.

Response:

```
{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "id": 25,
    "email": "rahul@example.com",
    "first_name": "Rahul",
    "last_name": "Sharma",
    "phone": "9876543210",
    "profile_image": null,
    "role": "STUDENT"
  }
}
```

```
}  
}
```

Student can only view their own profile.

13. Student Edit Profile

Endpoint:

```
PATCH /api/v1/student/profile/
```

Request:

```
{  
  "first_name": "Rahul",  
  "last_name": "Kumar",  
  "phone": "9876543210"  
}
```

Requirements:

- Authenticate user.
 - Ensure role is STUDENT.
 - Validate all fields.
 - Do not allow role modification.
 - Do not allow user ID modification.
 - Do not allow coach assignment modification from this endpoint.
 - Do not allow arbitrary fields to be updated.
-

14. Student Change Password

Endpoint:

```
POST /api/v1/student/change-password/
```

Request:

```
{  
  "old_password": "OldPassword123",  
  "new_password": "NewPassword123!"  
}
```

Requirements:

- Authentication required.
 - Verify old password.
 - Validate new password.
 - Hash password.
 - Never expose password data.
 - Prevent reuse of the current password.
 - Return a safe response.
-

15. Current User API

Create:

```
GET /api/v1/auth/me/
```

This API returns the authenticated user's basic information.

Example:

```
{
  "success": true,
  "data": {
    "id": 25,
    "email": "rahul@example.com",
    "first_name": "Rahul",
    "last_name": "Sharma",
    "role": "STUDENT"
  }
}
```

This will be used by the mobile application to restore the logged-in user's session.

16. JWT Authentication

Use:

```
django-rest-framework-simplejwt
```

Access tokens should have a relatively short lifetime.

For example:

Access Token: 15-30 minutes
Refresh Token: 7-30 days

Choose appropriate values based on security requirements and document them.

The mobile app will send:

Authorization: Bearer <access_token>

for protected APIs.

Never send JWT tokens in URLs.

17. Database Design

Use a custom Django User model.

Recommended:

```
User
----
id
email
password
first_name
last_name
phone
profile_image
role
is_active
is_staff
is_superuser
date_joined
created_at
updated_at
```

Role choices:

```
COACH
STUDENT
```

Email should be the primary login identifier.

Set:

```
USERNAME_FIELD = email
```

Use a custom UserManager.

18. Coach Assignment

Create a relationship that supports:

```
Student → Coach
```

For example:

```
StudentProfile
-----
id
user
coach
created_at
updated_at
```

or use a suitable relationship directly on the user/student model.

The relationship must support future multiple-coach functionality.

Current business rule:

```
There is one existing coach.
New students are assigned to that coach.
```

Do not hard-code:

```
coach_id = 1
```

Instead, retrieve the appropriate active coach from the database.

The implementation should be future-proof.

19. Security Requirements

Security is extremely important because the application will eventually contain private student videos and AI analysis.

Implement the following security practices now.

Authentication Security

- JWT authentication.
 - Secure password hashing using Django's password hashers.
 - Never store plaintext passwords.
 - Never return passwords.
 - Never return password hashes.
 - Generic login errors.
 - Account active/inactive validation.
 - Role validation.
 - Protected endpoints.
 - Token expiration.
 - Refresh-token handling.
-

20. Password Security

Use Django's password validation framework.

Require strong passwords.

At minimum validate:

- Minimum length.
- Not too common.
- Not entirely numeric.
- Not based on obvious user information.

Example acceptable password:

Wrestling@2026Secure

Do not implement your own insecure password hashing.

Use Django's built-in password hashing system.

21. Authorization

Authentication means:

Who are you?

Authorization means:

What are you allowed to do?

Both must be implemented.

Examples:

Student cannot access Coach Profile API.

Coach cannot edit a Student's profile through the Student profile API.

Student cannot modify their role.

Student cannot assign themselves to another coach.

Student cannot approve AI analysis.

Coach cannot modify another coach's profile.

Create reusable DRF permission classes.

For example:

IsCoach
IsStudent
IsOwner

Do not rely only on frontend restrictions.

Every authorization rule must be enforced server-side.

22. Object-Level Security

Never assume that because a user is authenticated they can access every resource.

For future APIs, enforce ownership.

For example:

Student A

must never be able to access:

Student B's videos
Student B's AI analysis
Student B's comments

Similarly, a coach should only access students assigned to that coach.

Design the permission architecture with this requirement in mind.

23. Input Validation

Validate all API inputs.

Validate:

- Email
- Phone
- Names
- Passwords
- IDs
- Uploaded files later
- JSON payloads

Do not trust mobile-app validation.

The backend must independently validate everything.

24. SQL Injection Protection

Use Django ORM.

Do not construct SQL queries using raw user input.

Avoid unsafe raw SQL.

If raw SQL is absolutely necessary, use parameterized queries.

25. XSS Protection

Validate and sanitize user-generated content.

This becomes particularly important later when:

- Coach comments

- AI suggestions
- Student notes

are introduced.

Never render raw HTML from users without sanitization.

26. CORS

Configure CORS securely.

During development, local development origins can be allowed.

Production should contain only approved application/API origins.

Do not use:

```
CORS_ALLOW_ALL_ORIGINS = True
```

in production.

27. CSRF

Understand that JWT APIs and browser session authentication have different CSRF requirements.

If using JWT through the Authorization header, configure authentication appropriately.

Do not disable security middleware globally just to make APIs work.

Keep Django security middleware enabled.

28. HTTPS

Production API must run over HTTPS.

Never send:

- Passwords
- JWT tokens
- Profile data

over plain HTTP in production.

Configure secure cookies if cookies are used anywhere.

29. Secrets

Never commit:

```
.env
database password
SECRET_KEY
JWT secret
AWS keys
OpenAI/API keys
storage credentials
email passwords
```

to Git.

Add:

```
.env
```

to:

```
.gitignore
```

Provide:

```
.env.example
```

with placeholder values.

Example:

```
SECRET_KEY=
DEBUG=False

DATABASE_NAME=
DATABASE_USER=
DATABASE_PASSWORD=
DATABASE_HOST=
DATABASE_PORT=

ALLOWED_HOSTS=
```

```
CORS_ALLOWED_ORIGINS=
```

30. Production Django Security

Configure appropriate production security settings.

For production:

```
DEBUG = False
```

Configure:

```
ALLOWED_HOSTS
SECURE_SSL_REDIRECT
SESSION_COOKIE_SECURE
CSRF_COOKIE_SECURE
SECURE_HSTS_SECONDS
SECURE_CONTENT_TYPE_NOSNIFF
SECURE_BROWSER_XSS_FILTER
X_FRAME_OPTIONS
```

Use settings appropriate to the deployment environment.

Do not blindly enable settings that could break local development.

Use separate development/production configuration where appropriate.

31. Rate Limiting

Authentication endpoints must have throttling/rate limiting.

Especially:

```
/login
/register
/change-password
/token/refresh
```

This protects against:

- Brute-force attacks

- Credential stuffing
- Automated registration
- Abuse

Use Django REST Framework throttling or a production-grade rate-limiting mechanism.

32. Login Enumeration Protection

Do not reveal whether an account exists.

Bad:

```
Email does not exist.
```

Better:

```
Invalid email or password.
```

The same principle should be used where appropriate for authentication-related responses.

33. User Enumeration

Registration errors should be carefully considered so the API does not unnecessarily reveal sensitive account information.

Use consistent and safe responses where appropriate.

34. Logging Security

Never log:

```
password
JWT token
refresh token
API secret
database password
```

Logs should contain useful diagnostic information without exposing credentials.

35. Error Handling

Create standardized error responses.

Success:

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": {}
}
```

Validation error:

```
{
  "success": false,
  "message": "Validation failed",
  "errors": {
    "email": [
      "Enter a valid email address."
    ]
  }
}
```

Authentication error:

```
{
  "success": false,
  "message": "Invalid email or password."
}
```

Server error:

```
{
  "success": false,
  "message": "An unexpected error occurred."
}
```

Do not expose Python stack traces to API clients in production.

36. API Status Codes

Use appropriate HTTP status codes.

Examples:

```
200 OK
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
429 Too Many Requests
500 Internal Server Error
```

Do not return HTTP 200 for every error.

37. Database Integrity

Use database constraints wherever appropriate.

Examples:

- Unique email
- Valid role choices
- Foreign key constraints
- Proper indexes
- Required fields

Use migrations properly.

Never manually modify production database tables without migrations.

38. API URL Structure

Use versioning.

Base:

```
/api/v1/
```

Authentication:

```
/api/v1/auth/coach/login/
/api/v1/auth/student/register/
/api/v1/auth/student/login/
```

```
/api/v1/auth/token/refresh/  
/api/v1/auth/me/
```

Coach:

```
/api/v1/coach/profile/  
/api/v1/coach/change-password/
```

Student:

```
/api/v1/student/profile/  
/api/v1/student/change-password/
```

39. Project Structure

Use a clean architecture.

Recommended:

```
wrestling_guide/  
|  
├─ manage.py  
|  
├─ config/  
|   │   ├── settings/  
|   │   │   ├── base.py  
|   │   │   ├── development.py  
|   │   │   └── production.py  
|   │   ├── urls.py  
|   │   ├── wsgi.py  
|   │   └── asgi.py  
|  
├─ accounts/  
|   ├── admin.py  
|   ├── apps.py  
|   ├── models.py  
|   ├── managers.py  
|   ├── serializers.py  
|   ├── views.py  
|   ├── permissions.py  
|   ├── urls.py  
|   ├── throttles.py  
|   ├── tests/  
|   └── migrations/  
|
```

```
├── students/
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   ├── permissions.py
│   ├── urls.py
│   └── tests/
├── coaches/
│   ├── models.py
│   ├── serializers.py
│   ├── views.py
│   ├── permissions.py
│   ├── urls.py
│   └── tests/
├── common/
│   ├── responses.py
│   ├── exceptions.py
│   ├── permissions.py
│   └── utils.py
├── requirements.txt
├── .env.example
├── .gitignore
└── README.md
```

The exact structure can be adjusted if there is a better Django architecture, but keep responsibilities separated.

40. Admin Panel

Configure Django Admin so the backend administrator can manage:

```
Users
Students
Coaches
```

The existing coach should be manageable from Django Admin.

The admin should be able to:

- View users
- Activate/deactivate users
- View student/coach roles
- View student-coach assignment
- Update profile information where appropriate

Do not expose Django admin APIs to the mobile application.

41. Existing Coach

There is already one coach in the database.

Do not create a public coach registration API.

If the database is empty during local development, provide a management command or documented seed process to create the initial coach.

For example:

```
python manage.py create_initial_coach
```

The command should securely request/set the required credentials and should not hard-code a production password.

42. Testing Requirements

Write automated tests.

At minimum test:

Authentication

- Student registration succeeds.
- Student registration rejects invalid email.
- Student registration rejects duplicate email.
- Student registration hashes password.
- Student cannot register as coach.
- Student login succeeds.
- Student login fails with wrong password.
- Coach login succeeds.
- Student cannot use coach login.
- Coach cannot use student login.
- Inactive users cannot login.
- JWT refresh works.

Authorization

- Unauthenticated users cannot access protected endpoints.
- Student cannot access coach profile.
- Coach cannot access student profile API.
- Student cannot change their role.

- Student cannot modify another user's data.

Profile

- Student can view own profile.
- Student can update own profile.
- Coach can view own profile.
- Coach can update own profile.
- Invalid profile data is rejected.

Password

- Correct old password is required.
- Incorrect old password is rejected.
- New password is validated.
- Password is hashed.
- Current password cannot be reused.

43. API Documentation

Every API must be documented.

Swagger should clearly show:

Endpoint
HTTP method
Authentication requirement
Request parameters
Request body
Response
Validation errors
HTTP status codes

Example:

POST /api/v1/auth/student/login/

The mobile team should be able to use Swagger to understand the entire API without reading backend source code.

44. README

Create a complete README containing:

Installation

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Environment setup

```
cp .env.example .env
```

Explain all environment variables.

Database

Explain PostgreSQL setup.

Migrations

```
python manage.py makemigrations
python manage.py migrate
```

Run server

```
python manage.py runserver
```

Create admin

```
python manage.py createsuperuser
```

Create initial coach

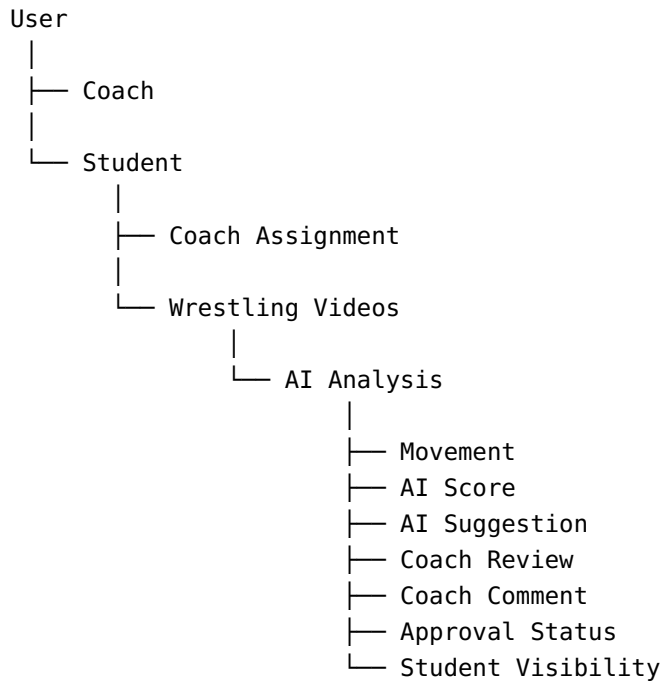
Document the management command/seed procedure.

API Documentation

Document the Swagger URL.

45. Future Database Design

Even though future functionality is NOT being implemented today, keep the architecture ready for:



Future entities may include:

```
WrestlingVideo
MovementAnalysis
AIAnalysis
CoachReview
CoachComment
```

Do not implement these today unless required for architecture.

46. Future Video Security

When video upload is implemented in a later phase, videos will contain private student content.

The architecture should therefore support:

- Private storage
- Signed URLs
- Access-controlled downloads
- File type validation
- File size limits
- Malware scanning where appropriate
- Video ownership checks
- Coach/student authorization
- No public video URLs
- No direct unauthenticated access

Do NOT simply place student videos in a public `/media/` directory.

47. Future AI Security

The future AI processing system must ensure:

Student A

cannot access:

Student B's AI analysis.

AI processing should be associated with the correct:

student
video
movement
analysis
coach

All AI-generated suggestions should be treated as application data, not trusted executable content.

48. Future Coach Approval Rule

The final system must enforce:

```
AI Analysis
  ↓
PENDING
  ↓
Coach Review
  ↓
APPROVED
  ↓
Student can view
```

Student must NOT see unapproved analysis.

This must be enforced in backend APIs, not only hidden in the mobile UI.

49. Important Development Rule

Do not build unnecessary functionality.

Today's implementation is:

```
AUTHENTICATION
+
PROFILE MANAGEMENT
+
PASSWORD MANAGEMENT
+
ROLE/PERMISSION SYSTEM
+
DATABASE FOUNDATION
+
API DOCUMENTATION
+
SECURITY
+
TESTING
```

Do NOT implement today:

```
✗ Flutter app
✗ Mobile UI
✗ Video upload
✗ Video processing
✗ AI movement detection
✗ Pose estimation
✗ AI suggestions
✗ Coach AI approval
✗ Coach comments
✗ Student analysis screen
✗ Payment
✗ Notifications
```

These belong to future phases.

50. Final API Checklist

The completed backend must expose:

Coach

```
POST    /api/v1/auth/coach/login/

GET     /api/v1/coach/profile/
PATCH  /api/v1/coach/profile/
POST    /api/v1/coach/change-password/
```

Student

```
POST    /api/v1/auth/student/register/
POST    /api/v1/auth/student/login/

GET     /api/v1/student/profile/
PATCH  /api/v1/student/profile/
POST    /api/v1/student/change-password/
```

Common

```
POST    /api/v1/auth/token/refresh/
GET     /api/v1/auth/me/
```

Documentation

```
GET     /api/docs/
GET     /api/schema/
```

51. Definition of Done

Today's backend task is considered complete only when:

- [] Django project is created.
- [] PostgreSQL is configured.
- [] Custom User model is implemented.
- [] Email is used for authentication.
- [] COACH and STUDENT roles are implemented.
- [] Existing coach can log in.
- [] No public coach registration endpoint exists.
- [] Student registration works.
- [] Student login works.
- [] JWT authentication works.
- [] JWT refresh works.
- [] Current-user API works.
- [] Coach profile API works.

- ☐ Coach edit profile works.
 - ☐ Coach password change works.
 - ☐ Student profile API works.
 - ☐ Student edit profile works.
 - ☐ Student password change works.
 - ☐ Student is assigned to the existing coach.
 - ☐ Role-based permissions are implemented.
 - ☐ Object-level authorization is implemented where applicable.
 - ☐ Authentication throttling/rate limiting is implemented.
 - ☐ Input validation is implemented.
 - ☐ Secure password hashing is implemented.
 - ☐ Sensitive information is not returned by APIs.
 - ☐ Secrets are stored in environment variables.
 - ☐ Production security settings are prepared.
 - ☐ CORS is configured correctly.
 - ☐ Standard API response format is implemented.
 - ☐ Proper HTTP status codes are used.
 - ☐ Automated tests are written.
 - ☐ Swagger/OpenAPI documentation is available.
 - ☐ README is complete.
 - ☐ `.env.example` is provided.
 - ☐ `.gitignore` is configured.
 - ☐ Initial coach creation/seed process is documented.
 - ☐ No mobile application code is created.
 - ☐ No AI/video processing is implemented yet.
-

52. Expected Deliverable

At the end of today's work, provide:

1. Complete Django backend project.
2. PostgreSQL database configuration.
3. Database migrations.
4. Authentication APIs.
5. Coach APIs.
6. Student APIs.
7. JWT authentication.
8. Role-based permissions.
9. Security configuration.
10. Automated tests.
11. Swagger/OpenAPI documentation.
12. `.env.example`.
13. `requirements.txt`.
14. Complete README.
15. Initial coach setup/seed instructions.
16. API endpoint list for the mobile team.

The backend should be production-ready in structure, secure by default, maintainable, and designed so that the future wrestling video/AI analysis system can be added without rewriting the authentication architecture.